Advanced Browsing Protection for Messenger

Emma Connor, Artem Ignatyev, **Kevin Lewi**

Outline

01 What is Advanced Browsing Protection?

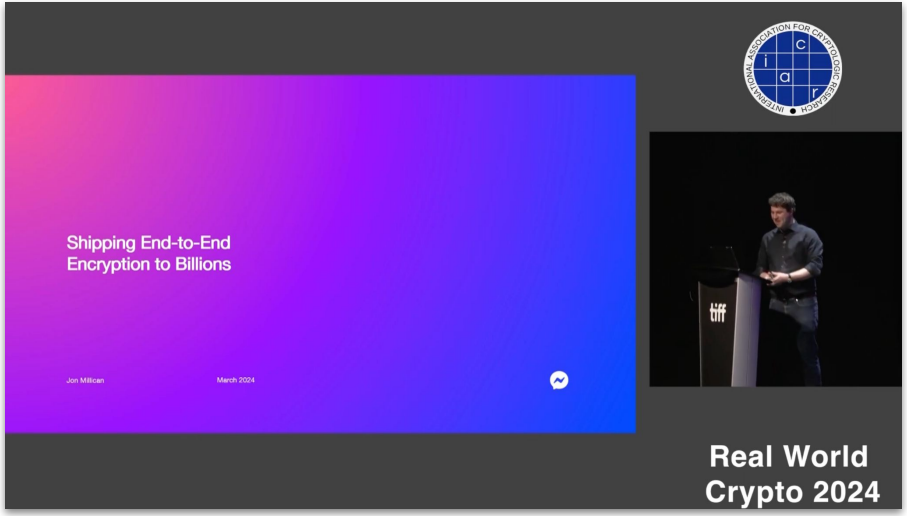
02 Private Information Retrieval

03 Prefix-Based Matching

04 TEEs, Attestation, and ORAM

05 Summary

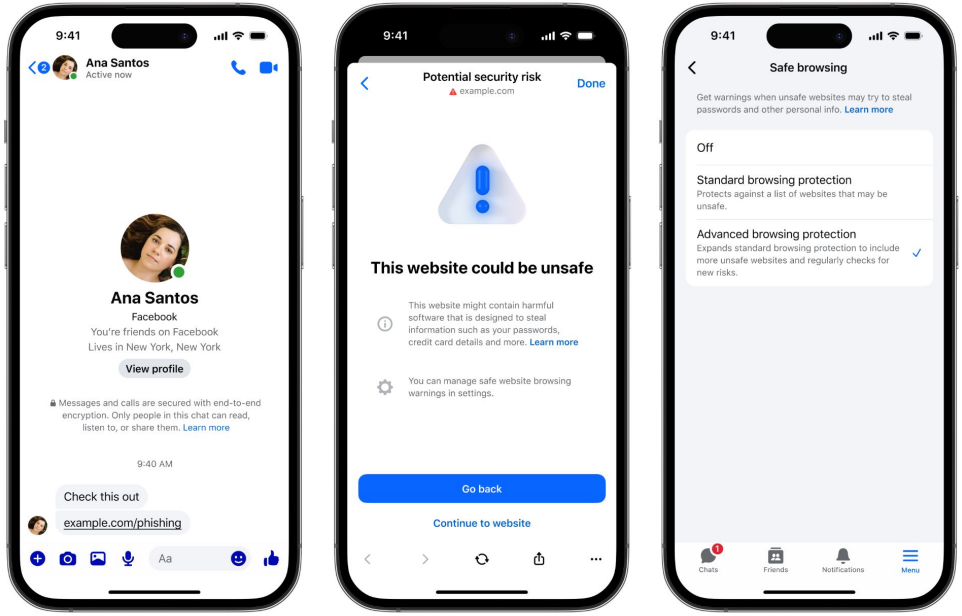
E2EE in Messenger



Advanced Browsing Protection

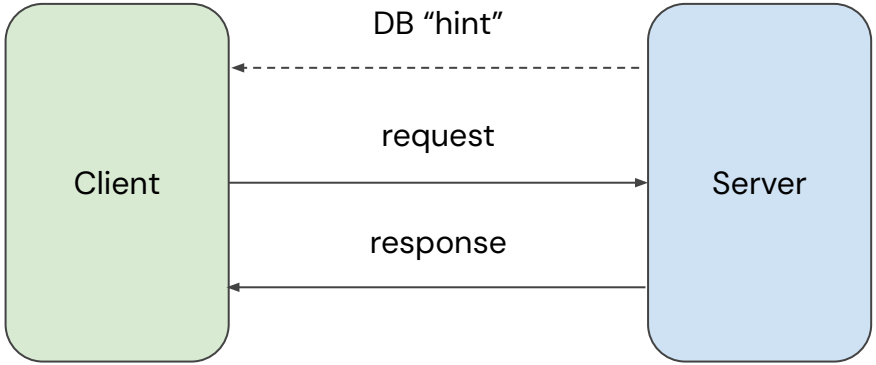
Protection against **malicious links** appearing in a user's chat threads

Triggered when a user explicitly clicks a link, and only when using the FB in-app browser



Basic PIR Scheme

Client clicked on link x



Server DB

malicious-link.com
tinyurl.com/yfxn7z
amz.co/x/site/...
...

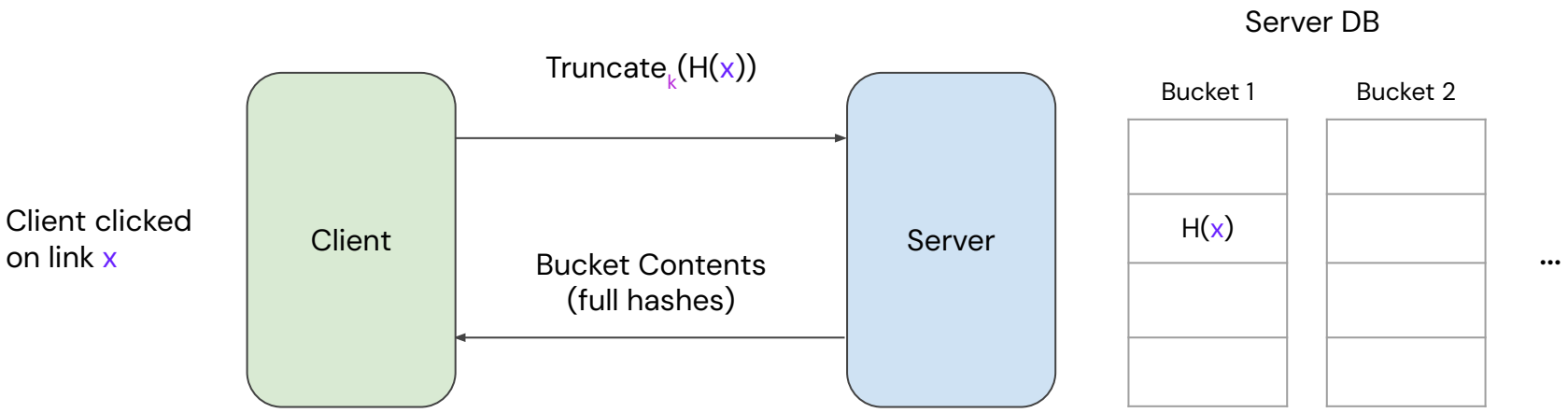
Client learns if x ∈ DB

Server learns as little as possible (ideally nothing)

Some Existing PIR Systems

- Google Safe Browsing
- Brave's FrodoPIR protocol
- Apple's Caller ID Lookup + URL checks (RWC 2025)
- Signal Private Contact Discovery

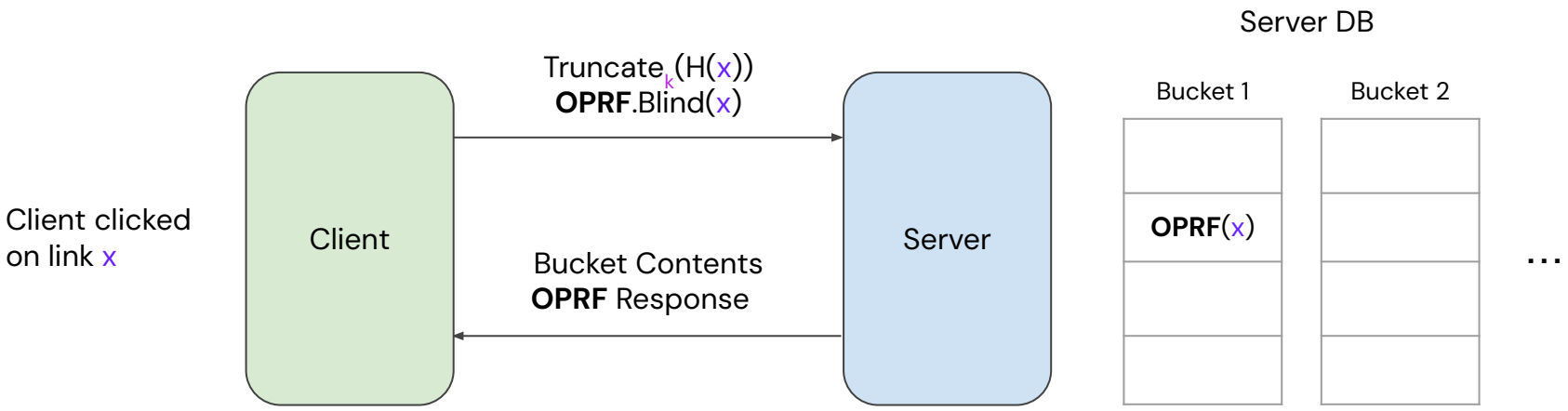
Basic PIR: Truncated Hash Prefixes



Client learns if $x \in$ DB
Client learns many DB entry hashes

Server learns k bits of H(x)

Preventing Offline Enumeration Using OPRFs [Thomas et al. 2017]



Client learns if $x \in$ DB

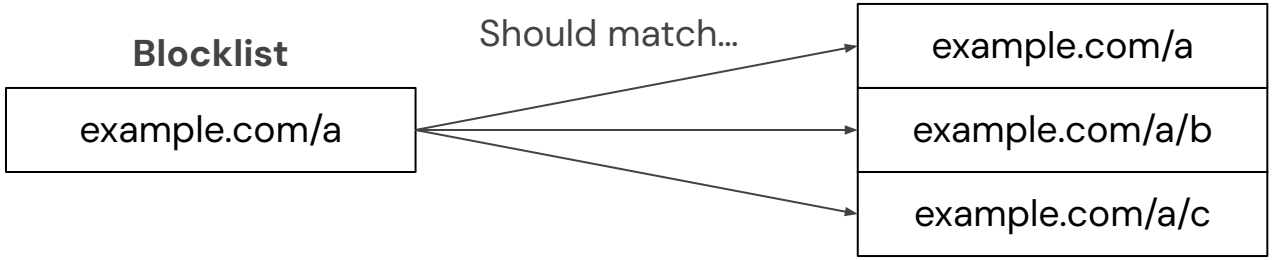
~~Client learns many DB entry hashes~~

Server learns k bits of H(x)

Prefix Matching?

What if we want to block entire...

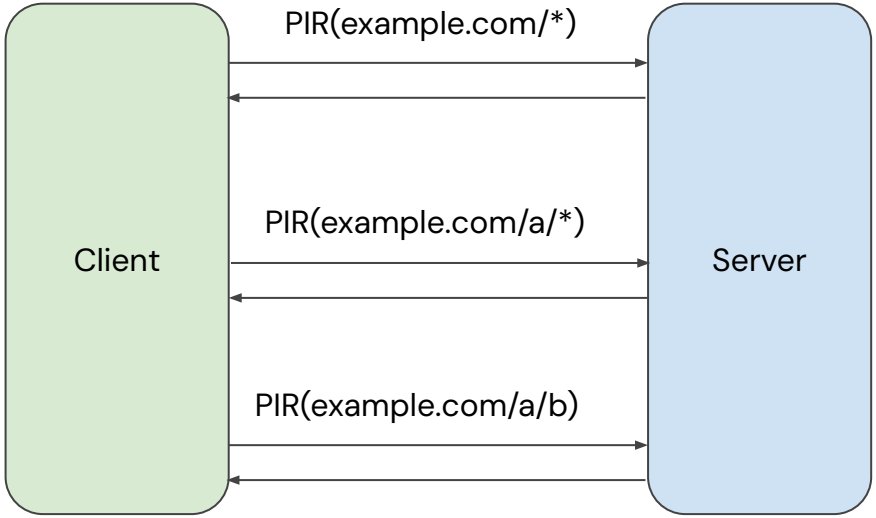
domains?
subdomains?
URL paths and children?



Prefix Matching: Attempt 1

Just query every segment!

Client clicked on link x



Server DB

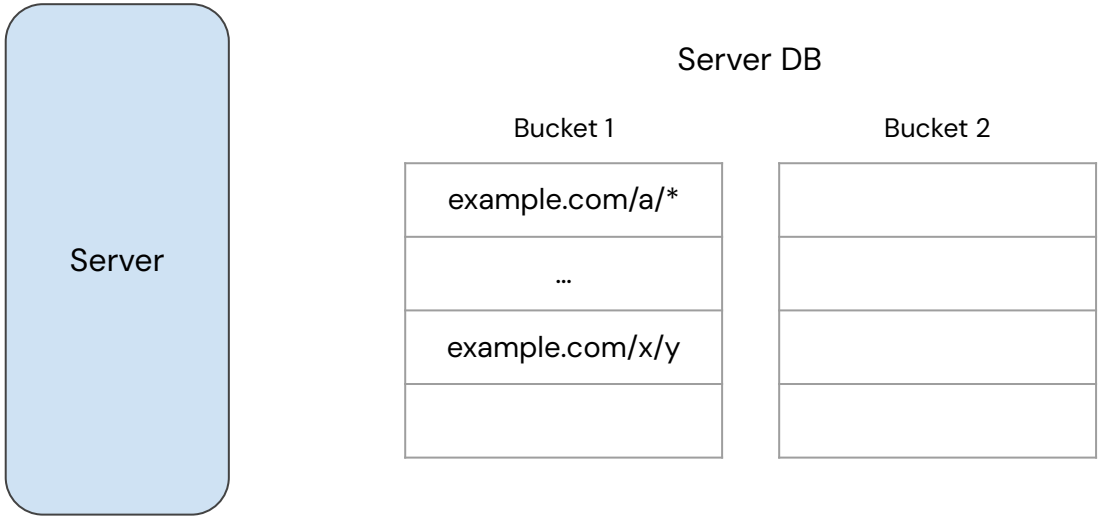
example.com/a/*
...

Client learns if x ∈ DB

Server learns k bits per bucket requested!

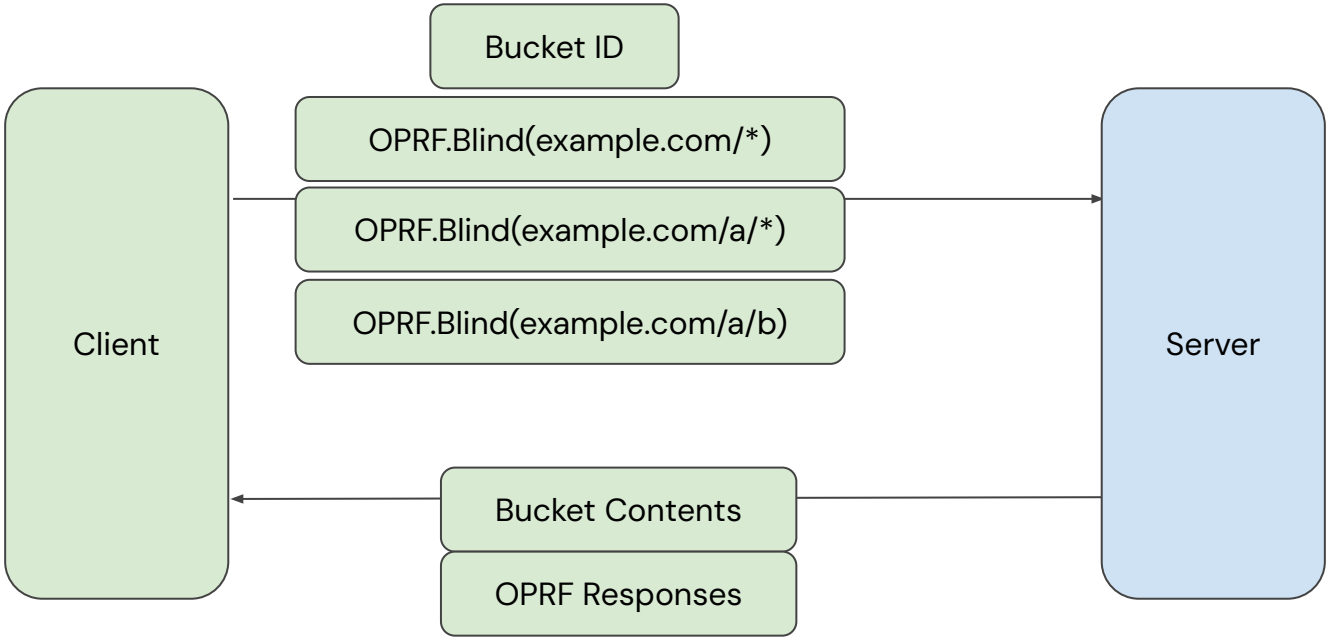
Prefix Matching: Attempt 2

Group related URLs into same bucket → client queries only 1 bucket



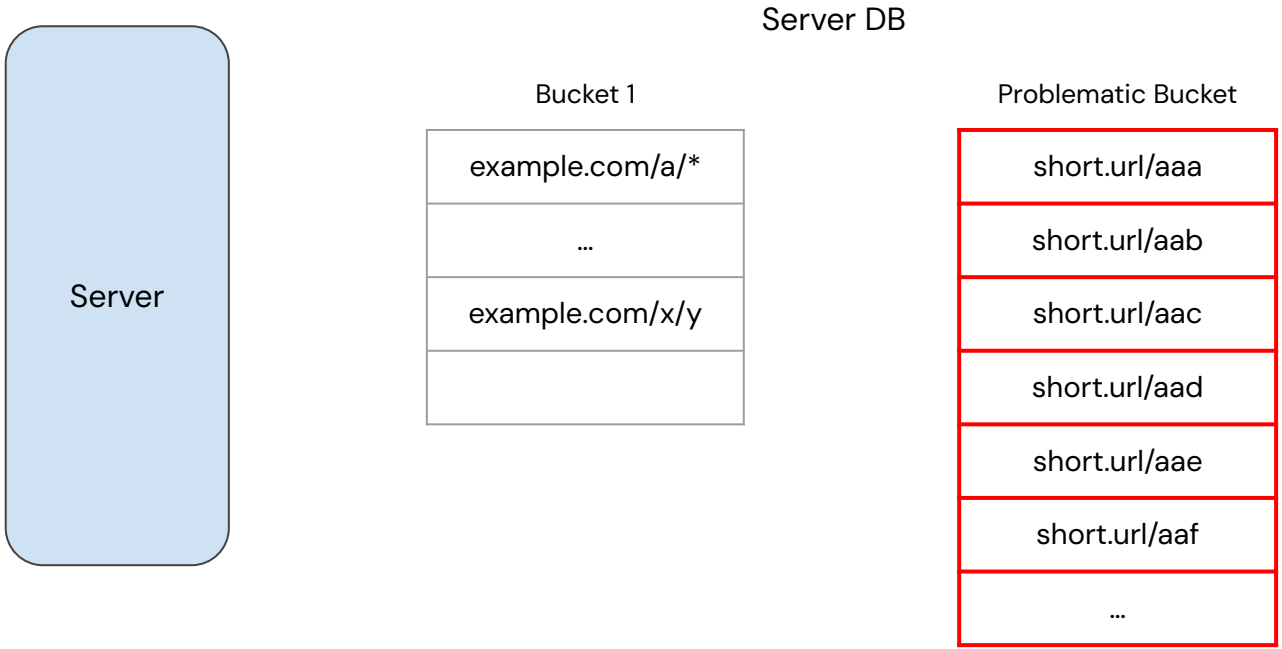
Prefix Matching: Attempt 2

Domain determines bucket ID



Prefix Matching: Attempt 2

This works… until we have an outlier domain



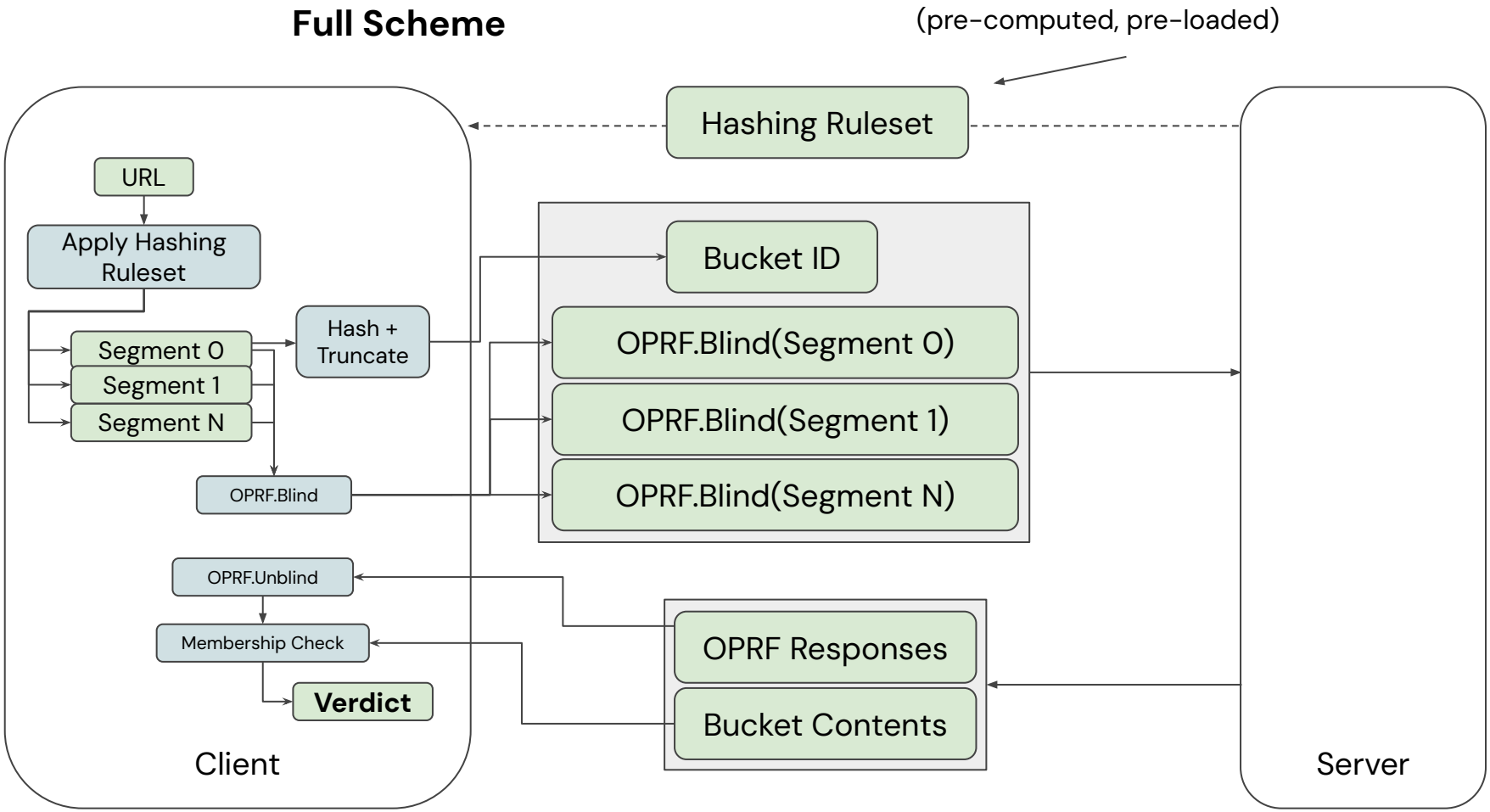
Prefix Matching: Attempt 3

“Hashing ruleset”: precomputed shared list, rules for deciding how much of the URL to hash to bucket ID

Hashing Ruleset

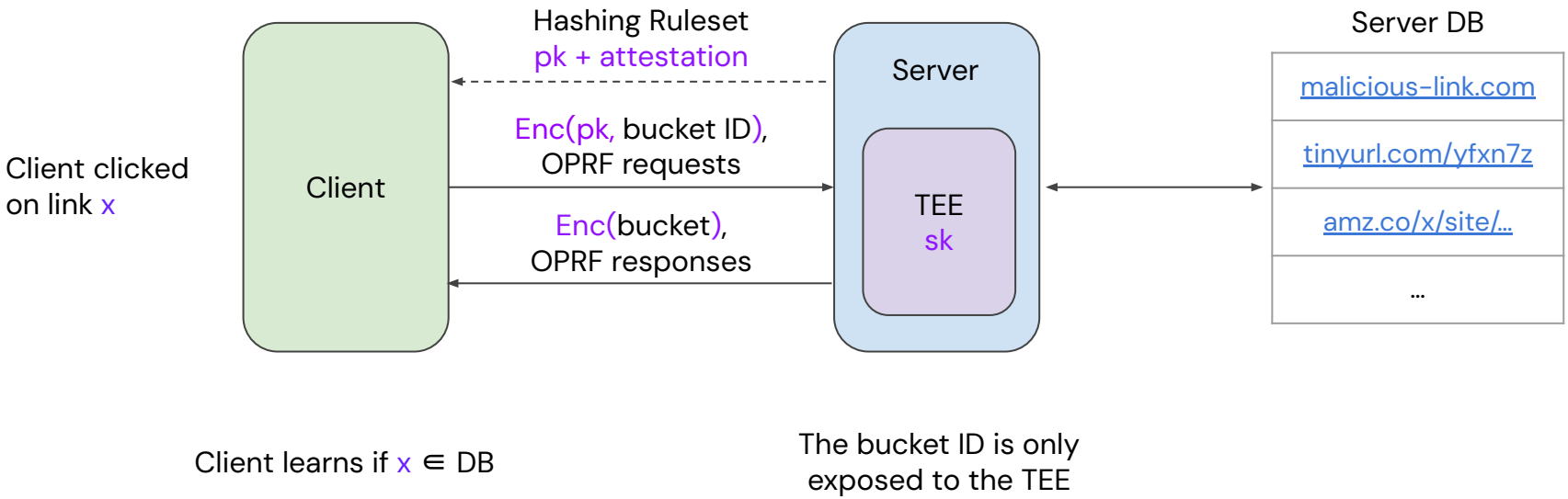
If hashed segment Is...	Then...
short.url	Add 1 Path Segment

Full Scheme

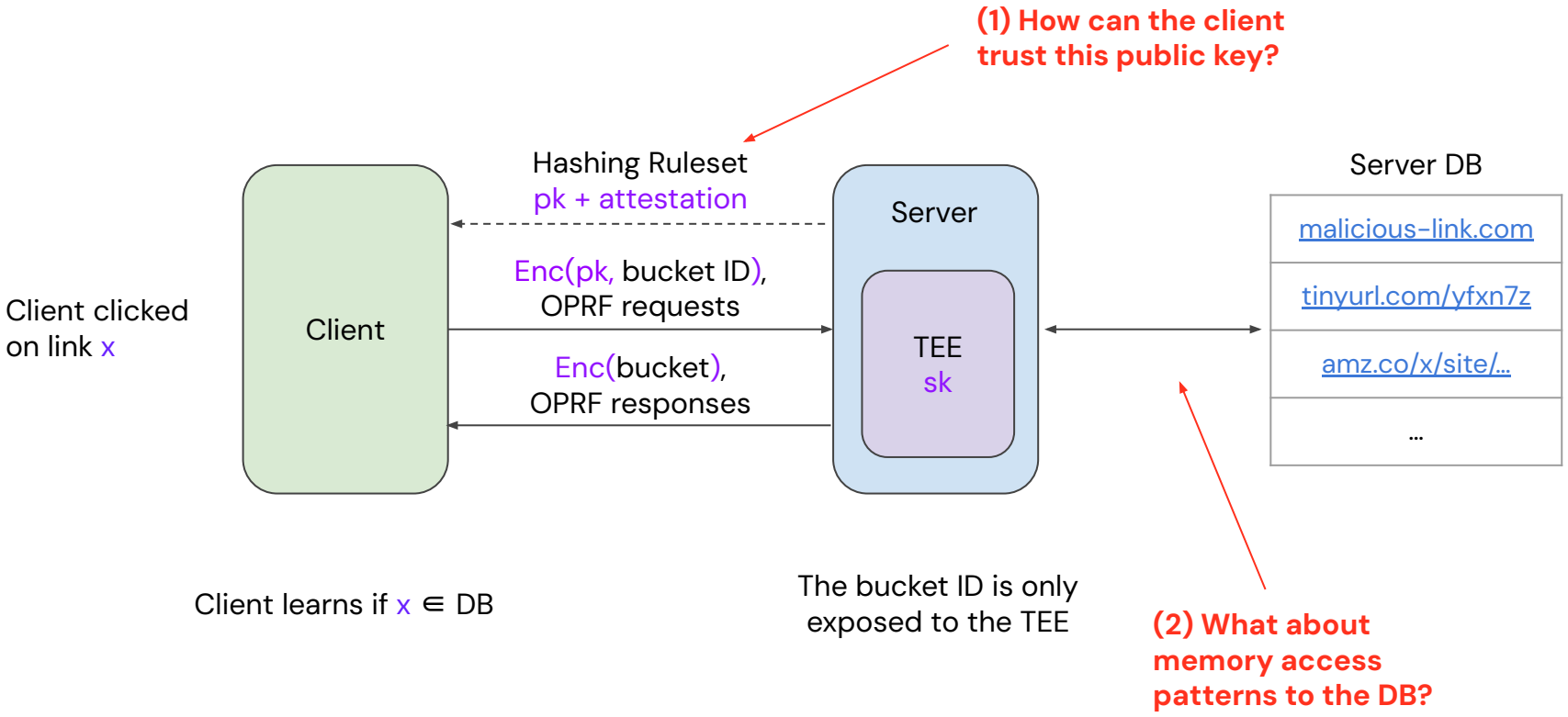

(pre-computed, pre-loaded)
Hashing Ruleset
URL
Apply Hashing Ruleset
Segment 0
Segment 1
Segment N
Hash + Truncate
OPRF.Blind
Bucket ID
OPRF.Blind(Segment 0)
OPRF.Blind(Segment 1)
OPRF.Blind(Segment N)
OPRF.Unblind
Membership Check
Verdict
OPRF Responses
Bucket Contents
Client
Server

Trusted Execution Environments

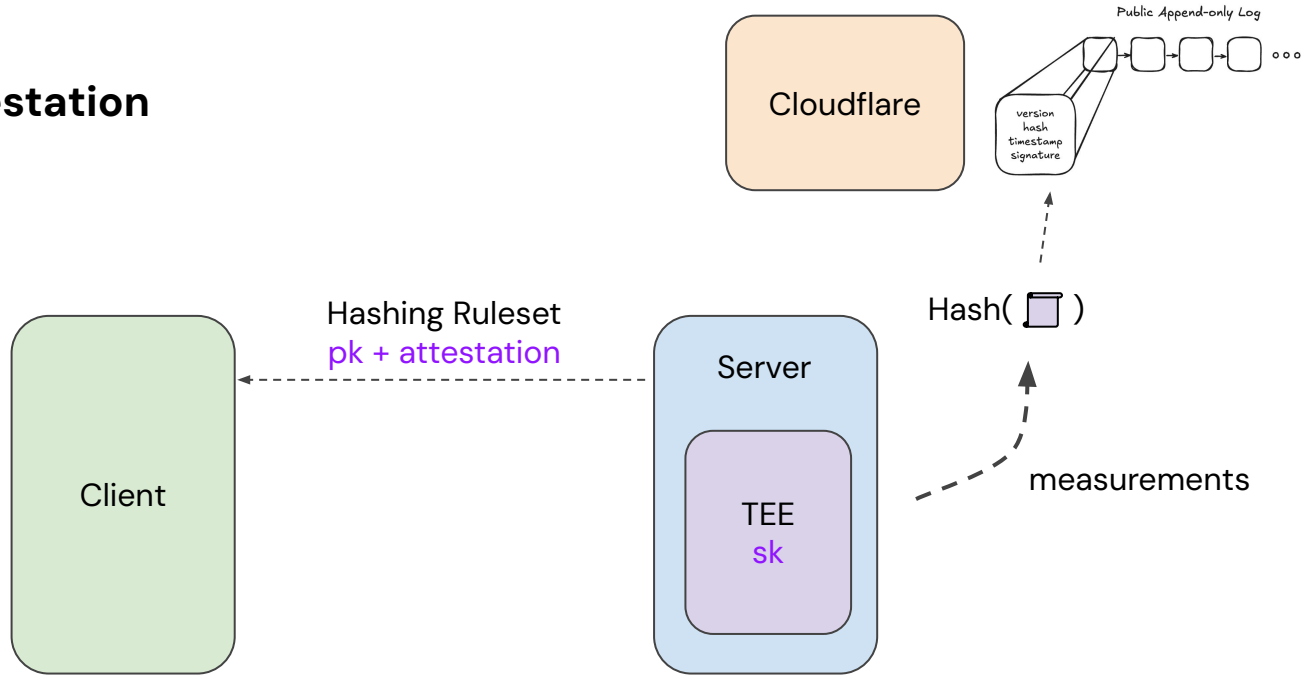
PIR with TEEs



PIR with TEEs



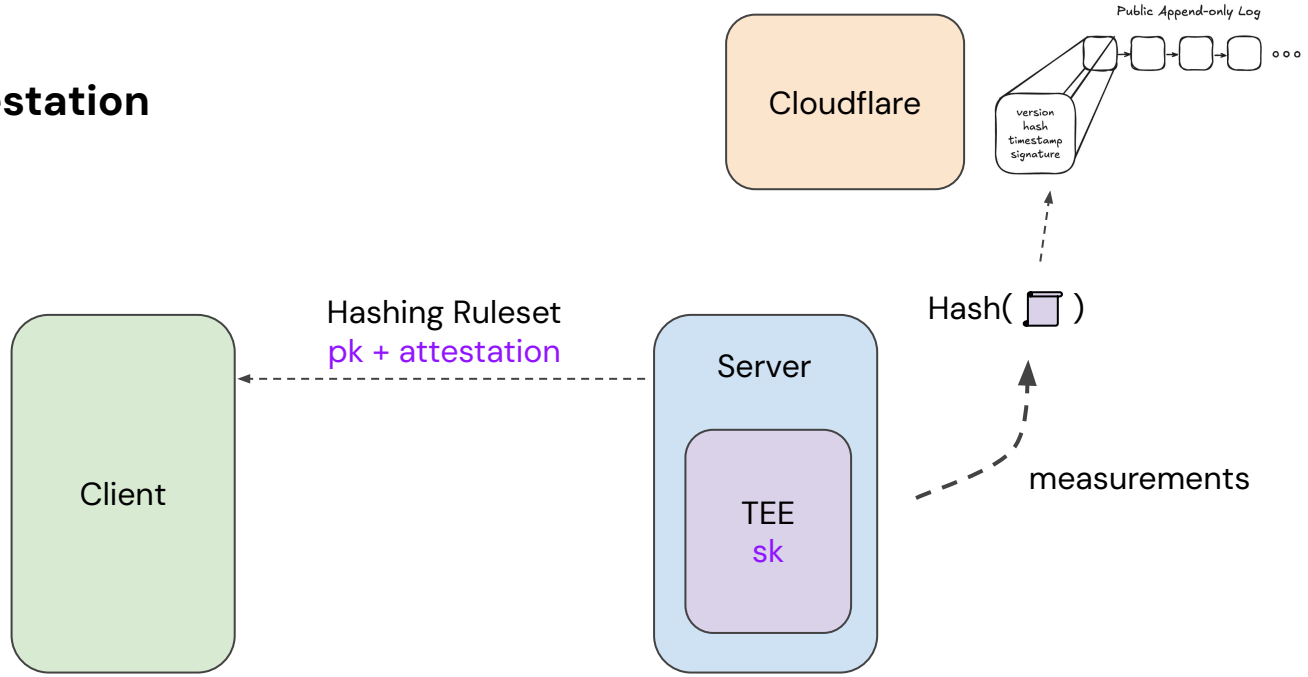
(1) Attestation



Report contains:

- Measurement hash
- pk
- Signature from AMD
- Transparency log signature

(1) Attestation



Report contains:

- Measurement hash
- pk
- Signature from AMD
- Transparency log signature

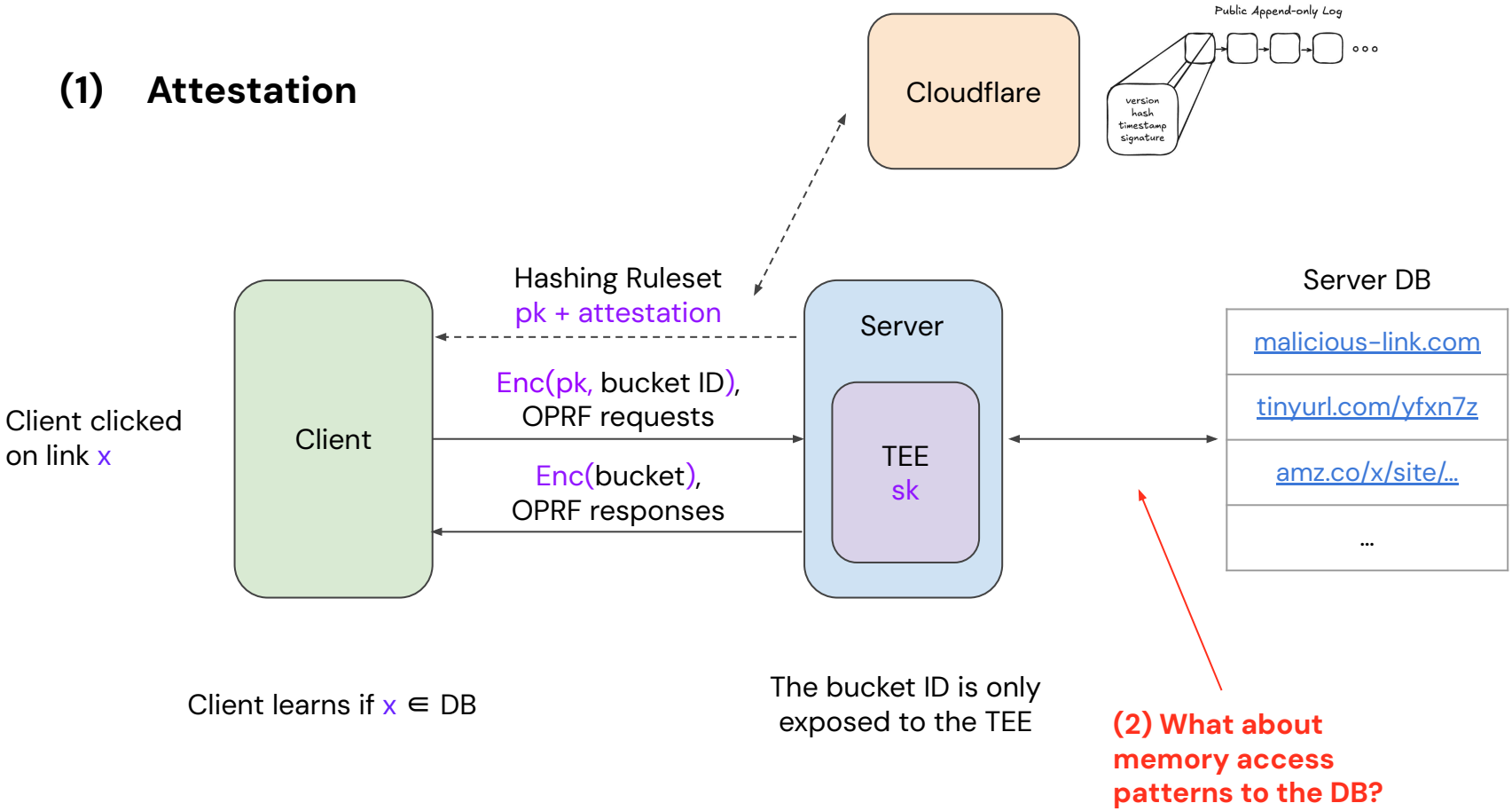
Commits to the application logic

Used to establish a secure channel

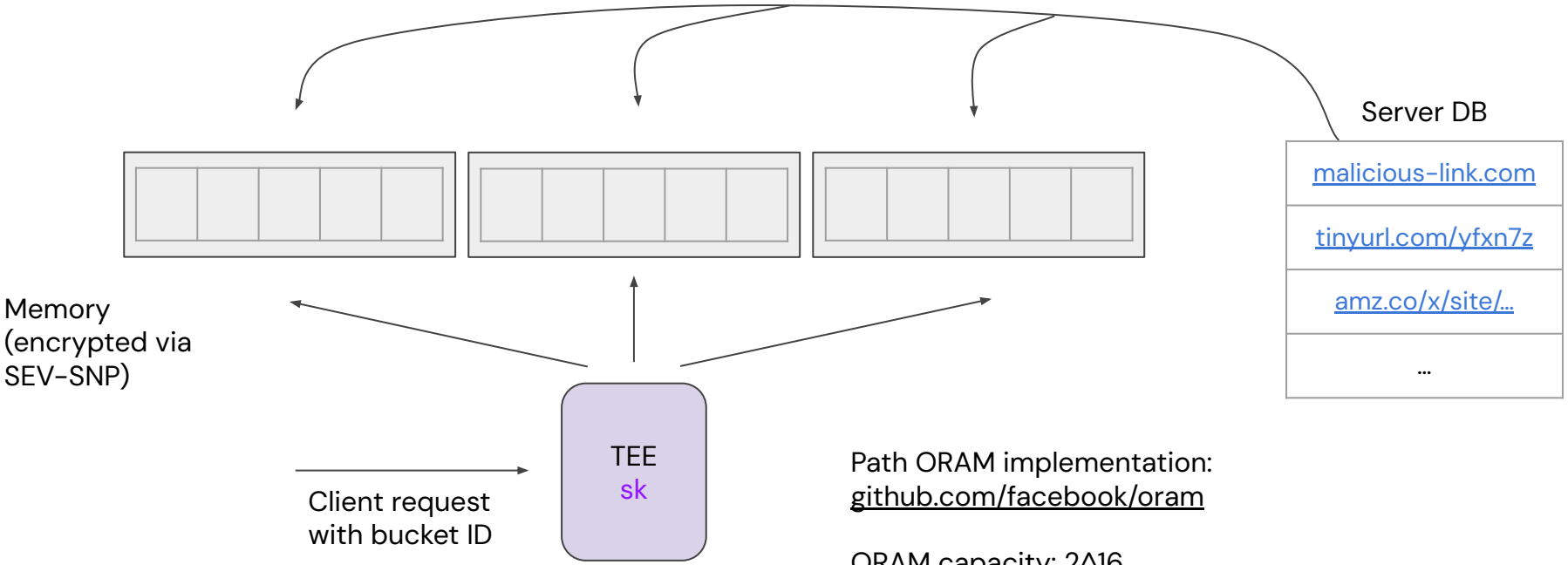
AMD SEV-SNP provided commitment to measurement

Mitigates server equivocation attacks

(1) Attestation



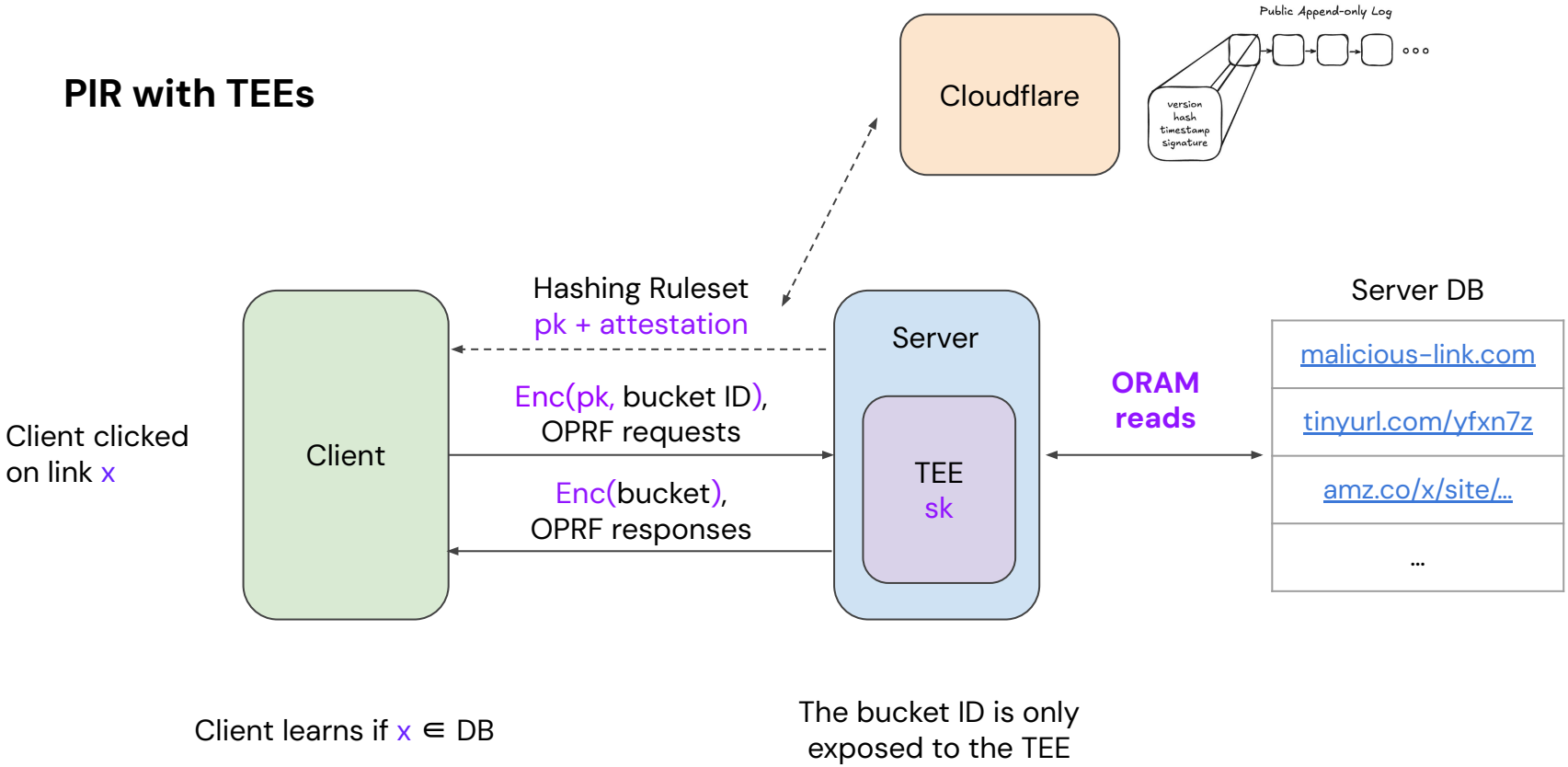
(2) Oblivious Memory Accesses



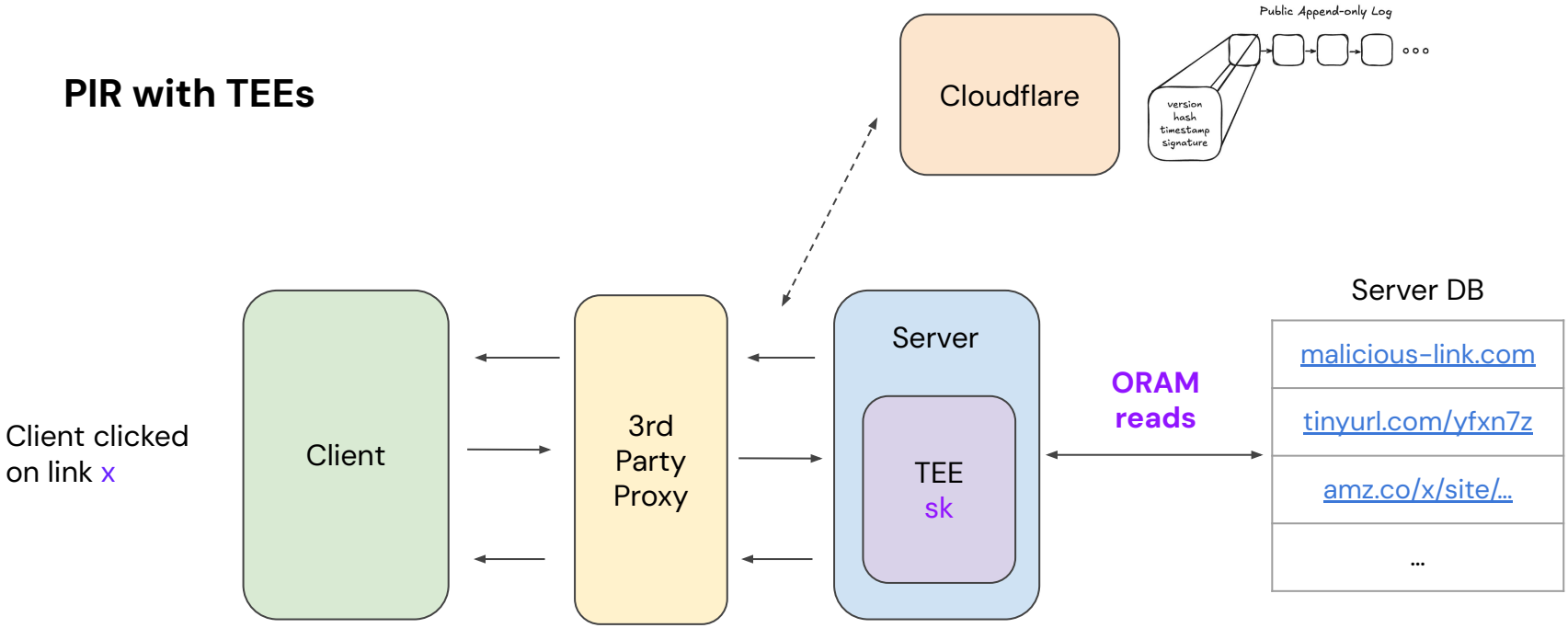
Path ORAM implementation:
github.com/facebook/oram

ORAM capacity: 2^16
block size: 16KB

PIR with TEEs



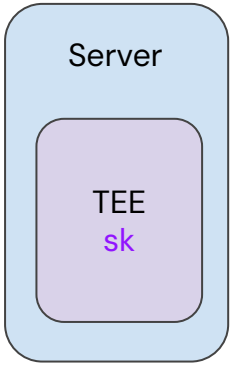
PIR with TEEs



Client learns if x ∈ DB

The bucket ID is only exposed to the TEE

Advanced Browsing Protection

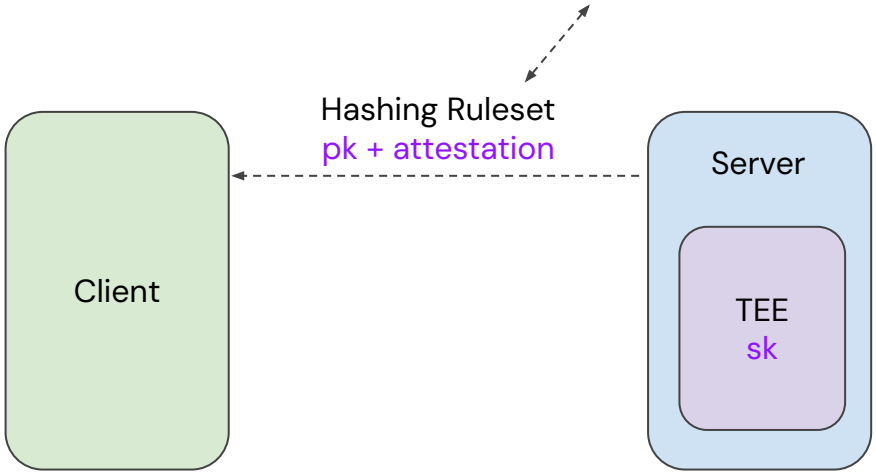


Server DB

malicious-link.com
tinyurl.com/yfxn7z
amz.co/x/site/...
...

Advanced Browsing Protection

If hashed segment Is...	Then...
short.url	Add 1 Path Segment
...	...

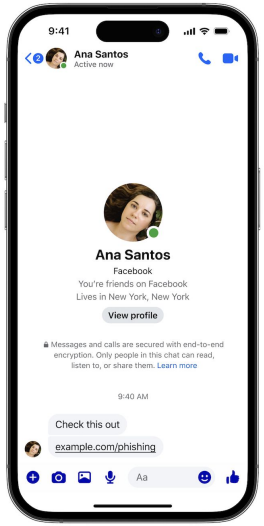


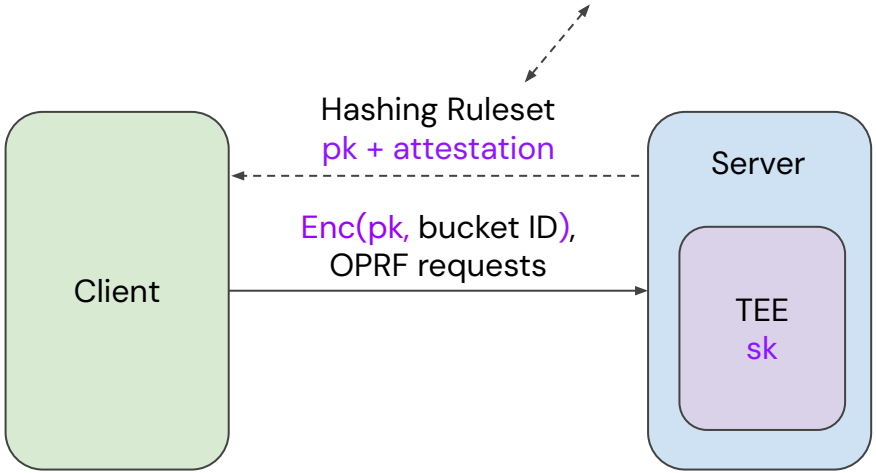
Server DB

malicious-link.com
tinyurl.com/yfxn7z
amz.co/x/site/...
...

Advanced Browsing Protection

If hashed segment Is...	Then...
short.url	Add 1 Path Segment
...	...



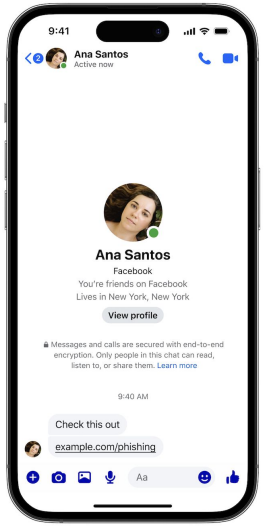


Server DB

malicious-link.com
tinyurl.com/yfxn7z
amz.co/x/site/...
...

Advanced Browsing Protection

If hashed segment Is...	Then...
short.url	Add 1 Path Segment
...	...



Client

Hashing Ruleset
pk + attestation

Enc(pk, bucket ID),
OPRF requests

Server

TEE
sk

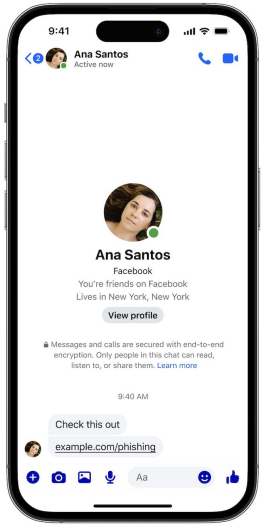
ORAM reads

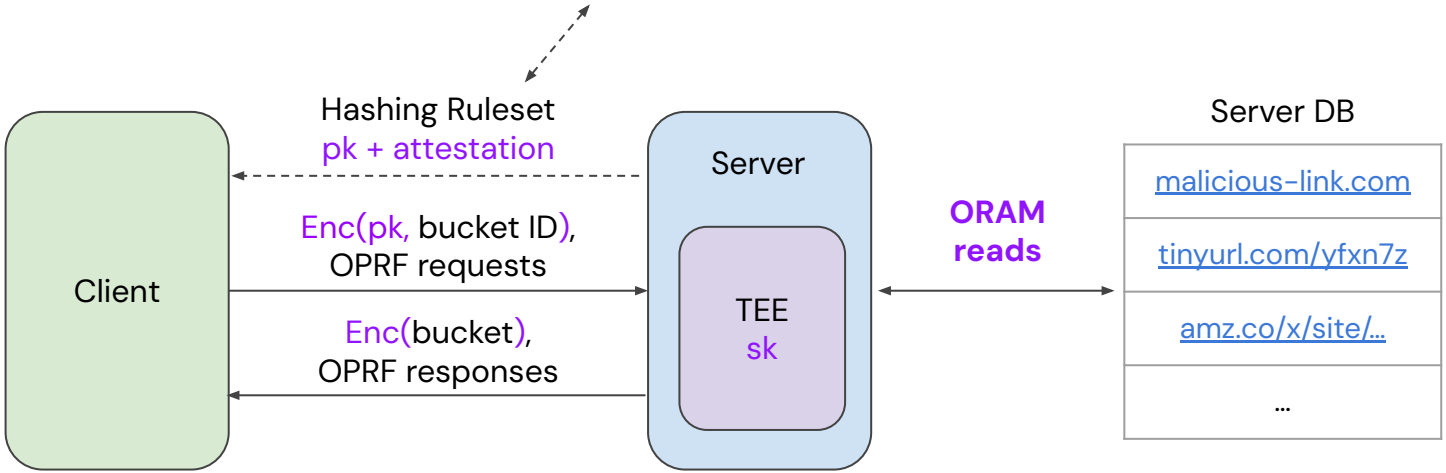
Server DB

malicious-link.com
tinyurl.com/yfxn7z
amz.co/x/site/...
...

Advanced Browsing Protection

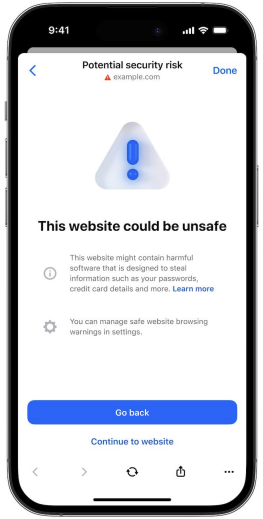
If hashed segment Is...	Then...
short.url	Add 1 Path Segment
...	...





Advanced Browsing Protection

If hashed segment Is...	Then...
short.url	Add 1 Path Segment
...	...



Client learns if x ∈ DB

Client

Hashing Ruleset
pk + attestation

Enc(pk, bucket ID),
OPRF requests

Enc(bucket),
OPRF responses

Server

TEE
sk

ORAM reads

Server DB

malicious-link.com
tinyurl.com/yfxn7z
amz.co/x/site/...
...

The bucket ID is only exposed to the TEE

Future Directions

- Can we further reduce our reliance on hardware security assumptions? E.g. shorter hash prefixes with reasonable bandwidth
- Can we improve the concrete efficiency of our ORAM usage, perhaps with Circuit ORAM?
- **We are open to suggestions on how we can continue to improve this system…**

Thanks!